Module 11: Memory and Resource Management

GOPAS

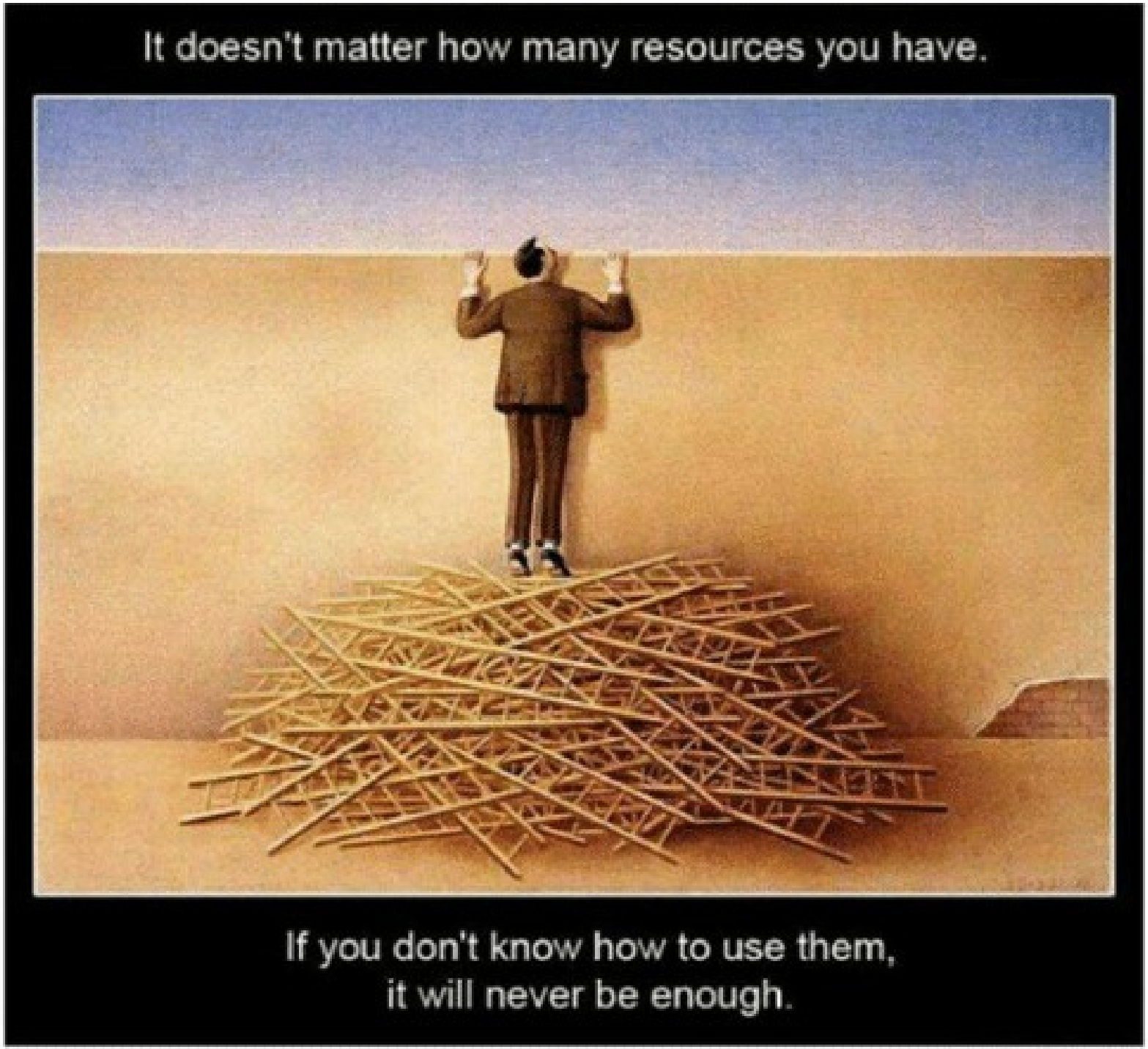
Overview

- Resource Management
- Garbage Collector
- Try Finally
- Using
- IDisposable Interface
- Destructors and Finalizers
- Weak Reference
- Generations

Developer Backgrounds

- C++
  - Manually use the new operator and delete operator
- COM
  - Manually implement reference counting and handle circular references in object
- .NET
  - Garbage Collerctor

GOPAS


It doesn't matter how many resources you have.
If you don't know how to use them,
it will never be enough.

Resource management

- **Implicit Resource Management** (Finalizer by GC)

```
class Foo {
          ~Foo()
          {
                    fs.Close();
          }
}
```

- **Explicit Resource Management** (Dispose by Program)
  a) Try..Finally
  b) Using Statement
  c) Using Declaration

Explicit resource release

- Try Finally

```
Resource r = new Resource();
try {
        r.Open();
}
finally
{
        if (r != null) ((IDisposable)r).Dispose();
}
```

- Using Statement

```
using (Resource r = new Resource()) { r.Open(); }
```

GOPAS

Using declarations >=C# 8.0

```
using (var r1 = new Resource1())
{
    using (var r2 = new Resource2())
    {
        using (var r3 = new Resource3())
        {

        }
    }
}
```


```
using var r1 = new Resource1();
using var r2 = new Resource2();
using var r3 = new Resource3();
```

Implicit resource management

- The order and timing of destruction is undefined
- Avoid destructors if possible
  - Performance costs
  - Complexity
  - Delay of memory resource release
- Finalize Code Called by Garbage Collection

```
class Foo
{
        ~Foo()
        {
                fs.Close();
        }
}
```

Implicit resource management

Finalizer is never guaranteed to be called

GOPAS

Controlling Garbage Collector

- To Force Garbage Collection

```
GC.Collect();
```

- To Suspend Calling Thread Until Thread's Queue of Finalizers Is Empty

```
GC.WaitForPendingFinalizers();
```

- To Request the System Not to Call the Finalizer Method

```
GC.SuppressFinalize(myObject);
```

Lab 11.1 - Finalizing a Garbage Collector

Exercise:

1. Interface IDisposable
2. Implicit Finalizing
3. Explicit Finalizing and design pattern

GOPAS

Weak Reference

- WeakReference objects can be collected even if they have a reference

```
string s = "ABCD";
WeakReference<string> wr = new WeakReference<string>(s);
s = null;
```

- **IsAlive - true** if the object referenced by the current object has not collected as garbage and is still accessible
- **Target** - Gets or sets the object reference

```
if (wr.TryGetTarget(out s))
{
    Console.WriteLine(s);
}
```

Generations

- The heap is organized into generations
- Three generations
  - Generation 0 - short-lived objects (Newly allocated objects)
  - Generation 2 - long-lived objects
- Collecting a generation means collecting objects in that generation and all its younger generations
- Generation 2 garbage collection is also known as a full garbage collection

```
GC.Collect(2);
```

Lab 11.2 - Weak Reference and Generations

Exercise:

1. Weak Reference
2. Generations

GOPAS

Review

- Resource Management
- Garbage Collector
- Try Finally
- Using
- IDisposable Interface
- Destructors and Finalizers
- Weak Reference
- Generations